

LogPulse AI - Incident Report

Generated : 2026-05-29 17:20:54 UTC

Trace ID : 2ae7171f-c951-40cc-8f4c-900dd6ae0676

1 - Incident Summary

Incident ID : 2ae7171f
Category : Memory
Severity : SeverityLevel.CRITICAL
Source : Kubernetes
Cache : MISS

2 - Raw Log

2026-05-29 16:10:10 FATAL worker-api OOMKilled: container exceeded memory limit 256Mi restarts=5
node=worker-3

3 - Classification

Category : Memory
Severity : SeverityLevel.CRITICAL
Summary : Kubernetes worker container killed for exceeding its memory limit.
Source : Kubernetes

4 - Diagnosis

Root Cause : The worker-api container's configured memory limit of 256Mi was insufficient for its operational workload, leading to Kubernetes terminating it due to Out Of Memory.
Confidence : 95%
Reasoning : The raw log explicitly states 'OOMKilled: container exceeded memory limit 256Mi', which is directly confirmed by the classification and the highly relevant retrieved playbook titled 'Pod OOMKilled Due to Insufficient Memory Limit'.
Source Tier : llm
Used History : No
Used Playbook : Yes

5 - Proposed Solution

Explanation:

Gemini unavailable; remediation derived from playbook 'Pod OOMKilled Due to Insufficient Memory Limit' (RAG confidence=0.7027). Review the playbook steps and adapt commands to the live environment before execution.

Steps:

- 1. Analyze historical application memory usage patterns using monitoring tools (e.g., Prometheus/Grafana, `kubectl top pod`). Identify the peak memory consumption.
- 2. Increase the `resources.limits.memory` for the affected container in its Deployment, StatefulSet, or Pod definition. It is often good practice to increase `resources.requests.memory` proportionally to ensure proper scheduling.
- 3. Apply the updated Kubernetes manifest to trigger a rollout and redeploy the affected pods.
- 4. Monitor the new deployment to confirm stable memory usage and absence of OOMKilled events.
- 5. name: my-app-container

Commands:

```
$ kubectl top pod
$ Apply the updated Kubernetes manifest to trigger a rollout and redeploy the affected pods.
$ kubectl apply -f deployment.yaml
```

6 - Risk Assessment

Risk Level : RISKLEVEL.SAFE

Safe Commands (3):

- [OK] kubectl top pod
- [OK] Apply the updated Kubernetes manifest to trigger a rollout and redeploy the affected pods.
- [OK] kubectl apply -f deployment.yaml

Flagged Commands (0):

(none)

7 - Operator Decision

Decision : OPERATORDECISION.APPROVED

Executed At : 2026-05-29T14:20:53.613695+00:00

Sandbox Results:

Command : kubectl top pod

Exit : -1

Time : 0ms

Output:

[sandbox unavailable -- docker not found]

Command : Apply the updated Kubernetes manifest to trigger a rollout and redeploy the affected pods.

Exit : -1

Time : 0ms

Output:

[sandbox unavailable -- docker not found]

Command : kubectl apply -f deployment.yaml

Exit : -1

Time : 0ms

Output:

[sandbox unavailable -- docker not found]

8 - Historical Context

Times Seen : 1
First occurrence -- no prior incidents.

9 - Incident Timeline

[14:20:53] WORKFLOW_COMPLETED -> Completed in 40270ms